EN4554 Assignment 1, Part 2: Tool-Calling Agent with LoRA

Group: Praveen V.V.J. (220491B), Sulojan R. (220626V)

This notebook builds a minimal tool-calling agent around a small LLM and measures how well the untouched model uses tools. It then LoRA-finetunes the model on synthetic tool-use conversations and measures it again on the same held-out test set.

Model	Qwen2.5-0.5B-Instruct
Tools	`calculator` , `unit_convert` , `get_weather` , `lookup_order`
Data	1,200 train / 100 test requests, template-generated
LoRA	r = 16, α = 32, all attention and MLP projections, loss on assistant turns only
Hardware	Colab T4, fp16 mixed precision, seed 42

Pipeline: agent loop → data → baseline evaluation → LoRA fine-tuning → LoRA evaluation → failure analysis.

0. Setup

These cells install the libraries and fix the random seed (42) for reproducibility. They also select the GPU and mount Google Drive, so the LoRA checkpoints and evaluation results survive a Colab disconnect. Colab's preinstalled `torchao` is removed because recent `peft` versions refuse to load alongside it.

```
!pip install -q "transformers>=4.46" "peft>=0.13" datasets accelerate
!pip uninstall -y -q torchao
```

```
WARNING: Skipping torchao as it is not installed.
```

```
import os, re, json, math, random, copy
from collections import Counter, defaultdict
import numpy as np
import torch
import pandas as pd
from tqdm.auto import tqdm
from transformers import AutoTokenizer, AutoModelForCausalLM, Trainer, TrainingArguments
from peft import LoraConfig, get_peft_model

SEED = 42
def set_seed(seed=SEED):
    random.seed(seed); np.random.seed(seed); torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
set_seed()
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

IN_COLAB = "google.colab" in str(get_ipython()) if "get_ipython" in globals() else False
if IN_COLAB:
    from google.colab import drive
    drive.mount("/content/drive")
    OUT_DIR = "/content/drive/MyDrive/EN4554_A1/part2"
else:
    OUT_DIR = "./outputs/part2"
os.makedirs(OUT_DIR, exist_ok=True)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive",
force_remount=True).
```

Experiment configuration

All the settings are in one place. The effective batch size is 16: a micro-batch of 2 with 8 gradient-accumulation steps. The micro-batch is small because Qwen's 152 k-token vocabulary makes the output logits big enough to run the T4 out of memory at batch 8.

```
CFG = dict(
    model_id   = "Qwen/Qwen2.5-0.5B-Instruct",
    n_train    = 1200,
    n_test     = 100,
    frac_no_tool = 0.2,
    frac_two_step = 0.1,
    max_steps  = 4,
    max_new_tokens = 128,
    max_len    = 768,
    r = 16, alpha = 32, dropout = 0.05,
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
    lr = 2e-4, epochs = 2, batch_size = 2, grad_accum = 8,
)
```

1. Tools

These are four plain Python functions the agent can call.

- **Made-up data:** `get_weather` and `lookup_order` return invented values (weather for six cities, 60 orders), so the model cannot answer from memory and has to call the tool.
- **Errors as strings:** every tool returns a string and reports problems as an `error: ...` message instead of raising, so a bad call never crashes the agent loop.
- **Safe calculator:** `calculator` accepts only arithmetic characters and runs with built-ins disabled.

```
WEATHER_DB = {
    "Kandy":  {"temp_c": 17, "condition": "foggy"},
    "Vavuniya":   {"temp_c": 29, "condition": "sunny"},
    "Ampara":  {"temp_c": 4,  "condition": "snow"},
    "Matara":  {"temp_c": 22, "condition": "light rain"},
    "Galle":  {"temp_c": 11, "condition": "windy"},
    "Jaffna":   {"temp_c": 33, "condition": "humid"},
}
ORDER_DB = {f"ORD-{1000+i}": {"status": random.choice(["shipped", "processing", "delivered", "cancelled"]),
                              "item": random.choice(["headphones", "desk lamp", "backpack", "keyboard", "water
bottle"]),
                              "eta_days": random.randint(0, 9)} for i in range(60)}

UNITS = {
    "length": {"m": 1, "km": 1000, "cm": 0.01, "mile": 1609.344, "ft": 0.3048, "inch": 0.0254},
    "mass":   {"kg": 1, "g": 0.001, "lb": 0.45359237, "oz": 0.028349523125},
}

def calculator(expression: str) -> str:
    if not re.fullmatch(r"[\d\s\.\+\-\*/\(\)%]+", expression):
        return "error: invalid expression"
    try:
        return str(round(eval(expression, {"__builtins__": {}}), 4))
    except Exception as e:
        return f"error: {e}"

def unit_convert(value: float, from_unit: str, to_unit: str) -> str:
    for dim in UNITS.values():
        if from_unit in dim and to_unit in dim:
            return str(round(float(value) * dim[from_unit] / dim[to_unit], 4))
    if {from_unit, to_unit} <= {"c", "f"}:
        v = float(value)
        return str(round(v * 9/5 + 32 if from_unit == "c" else (v - 32) * 5/9, 2))
    return "error: unsupported units"

def get_weather(city: str) -> str:
    w = WEATHER_DB.get(city.strip().title())
    return json.dumps(w) if w else "error: unknown city"

def lookup_order(order_id: str) -> str:
    o = ORDER_DB.get(order_id.strip().upper())
    return json.dumps(o) if o else "error: order not found"

TOOL_FNS = {f.__name__: f for f in [calculator, unit_convert, get_weather, lookup_order]}
```

Tool schemas

The model never sees the Python code. It sees only these JSON schemas (name, description, argument types), which the chat template renders into the system prompt. The system prompt itself is deliberately minimal, and the baseline, training and LoRA evaluation all use the same one.

```
TOOLS = [
    {"type": "function", "function": {"name": "calculator", "description": "Evaluate an arithmetic expression.",
        "parameters": {"type": "object", "properties": {"expression": {"type": "string", "description": "e.g.
(3+4)*2"}}, "required": ["expression"]}}},
    {"type": "function", "function": {"name": "unit_convert", "description": "Convert a value between units (m, km,
cm, mile, ft, inch, kg, g, lb, oz, c, f).",
        "parameters": {"type": "object", "properties": {"value": {"type": "number"}, "from_unit": {"type":
"string"}, "to_unit": {"type": "string"}},
                       "required": ["value", "from_unit", "to_unit"]}}},
    {"type": "function", "function": {"name": "get_weather", "description": "Get current weather for a city.",
        "parameters": {"type": "object", "properties": {"city": {"type": "string"}}, "required": ["city"]}}},
    {"type": "function", "function": {"name": "lookup_order", "description": "Look up an order by its ID (format
ORD-XXXX).",
        "parameters": {"type": "object", "properties": {"order_id": {"type": "string"}}, "required":
["order_id"]}}},
]
SYSTEM = "You are a helpful assistant. Use a tool when it is needed; otherwise answer directly."
```

2. Model and agent loop

This cell loads the tokenizer and Qwen2.5-0.5B-Instruct. The weights are kept in fp32, and generation runs under fp16 autocast. The printout shows the exact prompt the model receives. Qwen's chat template inserts the tool schemas and tells the model to answer with `<tool_call>{...}</tool_call>`.

```
tok = AutoTokenizer.from_pretrained(CFG["model_id"])
def load_model():
    return AutoModelForCausalLM.from_pretrained(CFG["model_id"], torch_dtype=torch.float32).to(DEVICE)
model = load_model()

print(tok.apply_chat_template([{"role": "system", "content": SYSTEM}, {"role": "user", "content": "hi"}],
                              tools=TOOLS, tokenize=False, add_generation_prompt=True))
```

```
<|im_start|>system
You are a helpful assistant. Use a tool when it is needed; otherwise answer directly.

# Tools

You may call one or more functions to assist with the user query.

You are provided with function signatures within <tools></tools> XML tags:
<tools>
{"type": "function", "function": {"name": "calculator", "description": "Evaluate an arithmetic expression.",
"parameters": {"type": "object", "properties": {"expression": {"type": "string", "description": "e.g. (3+4)*2"}},
"required": ["expression"]}}}
{"type": "function", "function": {"name": "unit_convert", "description": "Convert a value between units (m, km, cm,
mile, ft, inch, kg, g, lb, oz, c, f).", "parameters": {"type": "object", "properties": {"value": {"type":
"number"}, "from_unit": {"type": "string"}, "to_unit": {"type": "string"}}, "required": ["value", "from_unit",
"to_unit"]}}}
{"type": "function", "function": {"name": "get_weather", "description": "Get current weather for a city.",
"parameters": {"type": "object", "properties": {"city": {"type": "string"}}, "required": ["city"]}}}
{"type": "function", "function": {"name": "lookup_order", "description": "Look up an order by its ID (format ORD-
XXXX).", "parameters": {"type": "object", "properties": {"order_id": {"type": "string"}}, "required":
["order_id"]}}}
</tools>

For each function call, return a json object with function name and arguments within <tool_call></tool_call> XML
tags:
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call><|im_end|>
<|im_start|>user
hi<|im_end|>
<|im_start|>assistant
```

Agent loop

- `parse_tool_calls` extracts the `<tool_call>` blocks and flags malformed JSON, which feeds call validity.
- `execute` runs the named tool with the given arguments.
- `generate` produces one assistant turn with greedy decoding. Greedy decoding is deterministic, so the baseline and LoRA runs are directly comparable.
- `run_agent` loops: generate, parse, execute, and feed the result back as a `tool` message. It stops when the model answers in plain text or after 4 steps, and it returns a trace used for scoring.

```python
TOOL_CALL_RE = re.compile(r"<tool_call>\s*(.*?)\s*</tool_call>", re.S)

def parse_tool_calls(text):
    calls, valid = [], True
    blocks = TOOL_CALL_RE.findall(text)
    if text.count("<tool_call>") != len(blocks):
        valid = False
    for b in blocks:
        try:
            obj = json.loads(b)
        except json.JSONDecodeError:
            valid = False
            continue
        if isinstance(obj, dict) and isinstance(obj.get("name"), str) and isinstance(obj.get("arguments"), dict):
            calls.append({"name": obj["name"], "arguments": obj["arguments"]})
        else:
            valid = False
    return calls, valid

def execute(call):
    fn = TOOL_FNS.get(call["name"])
    if fn is None:
        return f"error: unknown tool {call['name']}"
    try:
        return fn(**call["arguments"])
    except Exception as e:
        return f"error: {e}"

@torch.no_grad()
def generate(model, messages):
    prompt = tok.apply_chat_template(messages, tools=TOOLS, tokenize=False, add_generation_prompt=True)
    enc = tok(prompt, return_tensors="pt").to(DEVICE)
    with torch.autocast("cuda", dtype=torch.float16):
        out = model.generate(**enc, max_new_tokens=CFG["max_new_tokens"], do_sample=False)
    return tok.decode(out[0, enc["input_ids"].shape[1]:], skip_special_tokens=True).strip()

def run_agent(model, user_msg, verbose=False):
    messages = [{"role": "system", "content": SYSTEM}, {"role": "user", "content": user_msg}]
    trace = dict(calls=[], valid_format=True, final=None, raw=[])
    for _ in range(CFG["max_steps"]):
        out = generate(model, messages)
        trace["raw"].append(out)
        calls, valid = parse_tool_calls(out)
        trace["valid_format"] &= valid
        if not calls:
            trace["final"] = out
            break
        messages.append({"role": "assistant", "content": "",
                         "tool_calls": [{"type": "function", "function": c} for c in calls]})
        for c in calls:
            result = execute(c)
            trace["calls"].append({**c, "result": result})
            messages.append({"role": "tool", "name": c["name"], "content": result})
        if verbose: print(out, "->", [c["result"] for c in trace["calls"][-len(calls):]])
    return trace

run_agent(model, "What's the weather in Kandy?", verbose=True)
```

```
<tool_call>
{"name": "get_weather", "arguments": {"city": "Kandy"}}
</tool_call> -> ['{"temp_c": 17, "condition": "foggy"}']
```

```
{'calls': [{'name': 'get_weather',
   'arguments': {'city': 'Kandy'},
   'result': '{"temp_c": 17, "condition": "foggy"}'}],
 'valid_format': True,
 'final': "The current temperature in Kandy is 17 degrees Celsius and it's foggy.",
 'raw': ['<tool_call>\n{"name": "get_weather", "arguments": {"city": "Kandy"}}\n</tool_call>',
  "The current temperature in Kandy is 17 degrees Celsius and it's foggy."]}
```

3. Data

Requests are generated from templates, and each gold answer is computed by actually running the tools. Each example has the form `{"user", "calls", "answer", "keys", "kind"}`:

- `calls` is the gold tool-call sequence, empty for no-tool requests.
- `keys` are the facts the final answer must contain.

Split	Size	Single-call	Two-call chains	No tool
Train	1,200	856	118	226
Test	100 (77 unique)	68	5	27

The test set is built from a **separate set of templates**, with different wording and sentence structure. An assertion checks that no test request also appears in training. In a two-call chain, the second call needs the first call's result, for example weather in °C, then a conversion to °F.

```
CITIES, ORDER_IDS = list(WEATHER_DB), list(ORDER_DB)

TRAIN_TEMPLATES = {
    "get_weather":  ["What's the weather in {city}?", "How warm is it in {city} right now?", "Is it raining in
{city}?",
                     "Tell me the current weather for {city}."],
    "lookup_order": ["Where is my order {oid}?", "Can you check the status of {oid}?", "Has {oid} shipped yet?",
                     "Look up order {oid} for me."],
    "calculator":   [("What is {a} times {b}?", "mul"), ("Compute {a} + {b} * {c}.", "add_mul"),
                     ("What's {a} divided by {b}?", "div"), ("What is {a} minus {b}?", "sub"), ("Add {a} and {b}.",
"add")],
    "unit_convert": ["Convert {v} {fu} to {tu}.", "How many {tu} is {v} {fu}?", "{v} {fu} in {tu}, please."],
    "no_tool":      ["What is the capital of France?", "Tell me a short joke.", "What does CPU stand for?", "Say
hello in Spanish.",
                     "What causes rain?", "Who painted the Mona Lisa?", "What is the largest planet in our solar
system?",
                     "Explain what a unit test is in one sentence.", "How many days are in a week?",
                     "What language is mostly spoken in Brazil?", "Give me a tip for studying better.",
                     "What is the chemical formula for water?"],
    "two_step":     ["What's the temperature in {city} in Fahrenheit?", "How many hours until order {oid}
arrives?",
                     "Give me the weather temperature of {city} in Fahrenheit, please.", "In how many hours will
{oid} be delivered?"],
}
TEST_TEMPLATES = {
    "get_weather":  ["Any idea what conditions are like over in {city} today?", "I'm travelling to {city} - should
I pack a coat?",
                     "{city} weather report, please.", "Could you tell me how the sky looks in {city}?"],
    "lookup_order": ["I placed {oid} last week; what's going on with it?", "Any update on {oid}?",
                     "Track {oid} for me.", "My package {oid} hasn't turned up - where is it at?"],
    "calculator":   [("Quick math: {a} multiplied by {b} please.", "mul"),
                     ("I need {a} split evenly into {b} groups; how many in each?", "div"),
                     ("Work out {a} plus {b} times {c} for me.", "add_mul"),
                     ("Take {b} away from {a} - what's left?", "sub"),
                     ("What do you get if you add {a} to {b}?", "add")],
    "unit_convert": ["I've got {v} {fu}; what's that in {tu}?", "Express {v} {fu} using {tu}.",
                     "Could you restate {v} {fu} as {tu}?", "What would {v} {fu} come to in {tu}?"],
    "no_tool":      ["Who wrote Romeo and Juliet?", "Which ocean is the largest?", "What does RAM stand for in
computing?",
                     "How do you say thank you in French?", "Why is the sky blue?", "Write me a one-line pep
talk.",
                     "What is the boiling point of water at sea level in Celsius?", "What's the tallest mountain on
Earth?",
                     "Name a primary colour.", "What is photosynthesis?", "Suggest a name for a pet goldfish."],
    "two_step":     ["Give me the {city} temperature but in °F.", "{oid} - roughly how many hours until it gets
here?",
                     "How hot is {city}, expressed in Fahrenheit?", "Hours left before {oid} lands at my door?"],
}
NO_TOOL_ANSWERS = {
    "What is the capital of France?": ("The capital of France is Paris.", ["paris"]),
    "Tell me a short joke.": ("Why do programmers prefer dark mode? Because light attracts bugs.", []),
    "What does CPU stand for?": ("CPU stands for Central Processing Unit.", ["central processing unit"]),
    "Say hello in Spanish.": ("Hello in Spanish is \"Hola\".", ["hola"]),
    "What causes rain?": ("Rain forms when water vapour in clouds condenses into droplets that grow heavy enough to
fall.", ["condens"]),
    "Who painted the Mona Lisa?": ("The Mona Lisa was painted by Leonardo da Vinci.", ["leonardo"]),
    "What is the largest planet in our solar system?": ("Jupiter is the largest planet in our solar system.",
["jupiter"]),
    "Explain what a unit test is in one sentence.": ("A unit test is a small automated check that verifies one
piece of code behaves as expected.", []),
    "How many days are in a week?": ("There are seven days in a week.", ["7|seven"]),
    "What language is mostly spoken in Brazil?": ("Portuguese is the main language spoken in Brazil.",
["portuguese"]),
    "Give me a tip for studying better.": ("Study in short focused sessions with regular breaks, and test yourself
instead of only rereading.", []),
    "What is the chemical formula for water?": ("The chemical formula for water is H2O.", ["h2o"]),
    "Who wrote Romeo and Juliet?": ("Romeo and Juliet was written by William Shakespeare.", ["shakespeare"]),
    "Which ocean is the largest?": ("The Pacific Ocean is the largest ocean.", ["pacific"]),
    "What does RAM stand for in computing?": ("RAM stands for Random Access Memory.", ["random access memory"]),
    "How do you say thank you in French?": ("Thank you in French is \"Merci\".", ["merci"]),
    "Why is the sky blue?": ("The sky looks blue because air molecules scatter short blue wavelengths of sunlight
more than red ones.", ["scatter"]),
```

```
    "Write me a one-line pep talk.": ("You have already survived every hard day so far - keep going.", []),
    "What is the boiling point of water at sea level in Celsius?": ("Water boils at 100 °C at sea level.",
["100"]),
    "What's the tallest mountain on Earth?": ("Mount Everest is the tallest mountain on Earth.", ["everest"]),
    "Name a primary colour.": ("Red is a primary colour.", ["red|blue|yellow"]),
    "What is photosynthesis?": ("Photosynthesis is how plants use sunlight, water and carbon dioxide to make sugar
and oxygen.", ["sunlight|light"]),
    "Suggest a name for a pet goldfish.": ("How about \"Bubbles\"?", []),
}

def _fmt(x):
    x = str(x)
    return x[:-2] if x.endswith(".0") else x

def sample_example(templates, rng):
    r = rng.random()
    if r < CFG["frac_no_tool"]:
        kind = "no_tool"
    elif r < CFG["frac_no_tool"] + CFG["frac_two_step"]:
        kind = "two_step"
    else:
        kind = rng.choice(["get_weather", "lookup_order", "calculator", "unit_convert"])

    if kind == "no_tool":
        user = rng.choice(templates["no_tool"])
        ans, keys = NO_TOOL_ANSWERS[user]
        return {"user": user, "calls": [], "answer": ans, "keys": keys, "kind": kind}

    if kind == "get_weather":
        city = rng.choice(CITIES)
        calls = [{"name": "get_weather", "arguments": {"city": city}}]
        w = json.loads(execute(calls[0]))
        ans, keys = f"It's {w['temp_c']}°C and {w['condition']} in {city}.", [str(w["temp_c"]), w["condition"]]
        user = rng.choice(templates[kind]).format(city=city)

    elif kind == "lookup_order":
        oid = rng.choice(ORDER_IDS)
        calls = [{"name": "lookup_order", "arguments": {"order_id": oid}}]
        o = json.loads(execute(calls[0]))
        ans = f"Order {oid} ({o['item']}) is {o['status']}; estimated delivery in {o['eta_days']} days."
        keys = [o["status"], o["item"]]
        user = rng.choice(templates[kind]).format(oid=oid)

    elif kind == "calculator":
        tpl, op = rng.choice(templates[kind])
        a, b, c = rng.randint(2, 99), rng.randint(2, 99), rng.randint(2, 50)
        if op == "div":
            b = rng.randint(2, 12); a = b * rng.randint(2, 50)
        elif op == "sub":
            a, b = max(a, b) + 1, min(a, b)
        expr = {"mul": f"{a}*{b}", "div": f"{a}/{b}", "add_mul": f"{a}+{b}*{c}", "sub": f"{a}-{b}", "add": f"{a}+
{b}"}[op]
        calls = [{"name": "calculator", "arguments": {"expression": expr}}]
        res = execute(calls[0])
        ans, keys = f"The result is {_fmt(res)}.", [res]
        user = tpl.format(a=a, b=b, c=c)

    elif kind == "unit_convert":
        dim = rng.choice(list(UNITS))
        fu, tu = rng.sample(list(UNITS[dim]), 2)
        v = rng.choice([rng.randint(1, 100), rng.randint(1, 100) + 0.5])
        calls = [{"name": "unit_convert", "arguments": {"value": v, "from_unit": fu, "to_unit": tu}}]
        res = execute(calls[0])
        ans, keys = f"{v} {fu} is {_fmt(res)} {tu}.", [res]
        user = rng.choice(templates[kind]).format(v=v, fu=fu, tu=tu)

    else:
        tpl = rng.choice(templates["two_step"])
        if "{city}" in tpl:
            city = rng.choice(CITIES)
            c1 = {"name": "get_weather", "arguments": {"city": city}}
            t = json.loads(execute(c1))["temp_c"]
```

```
            c2 = {"name": "unit_convert", "arguments": {"value": t, "from_unit": "c", "to_unit": "f"}}
            res = execute(c2)
            ans, keys, user = f"It's {_fmt(res)}°F in {city}.", [res], tpl.format(city=city)
        else:
            oid = rng.choice(ORDER_IDS)
            c1 = {"name": "lookup_order", "arguments": {"order_id": oid}}
            eta = json.loads(execute(c1))["eta_days"]
            c2 = {"name": "calculator", "arguments": {"expression": f"{eta}*24"}}
            res = execute(c2)
            ans, keys, user = f"Order {oid} should arrive in {_fmt(res)} hours.", [res], tpl.format(oid=oid)
        calls = [c1, c2]

    return {"user": user, "calls": calls, "answer": ans, "keys": keys, "kind": kind}

def build_split(templates, n, seed):
    rng = random.Random(seed)
    return [sample_example(templates, rng) for _ in range(n)]

train_set = build_split(TRAIN_TEMPLATES, CFG["n_train"], SEED)
test_set  = build_split(TEST_TEMPLATES,  CFG["n_test"],  SEED + 1)
assert not ({e["user"] for e in train_set} & {e["user"] for e in test_set}), "train/test leakage"
print(len(train_set), len(test_set))
print("train kinds:", Counter(e["kind"] for e in train_set)); print("test kinds: ", Counter(e["kind"] for e in
test_set))
train_set[:3]
```

```
1200 100
train kinds: Counter({'lookup_order': 231, 'no_tool': 226, 'calculator': 217, 'unit_convert': 210, 'get_weather':
198, 'two_step': 118})
test kinds:  Counter({'no_tool': 27, 'lookup_order': 20, 'get_weather': 19, 'unit_convert': 15, 'calculator': 14,
'two_step': 5})
```

```
[{'user': 'Is it raining in Jaffna?',
  'calls': [{'name': 'get_weather', 'arguments': {'city': 'Jaffna'}}],
  'answer': "It's 33°C and humid in Jaffna.",
  'keys': ['33', 'humid'],
  'kind': 'get_weather'},
 {'user': 'How many hours until order ORD-1047 arrives?',
  'calls': [{'name': 'lookup_order', 'arguments': {'order_id': 'ORD-1047'}},
   {'name': 'calculator', 'arguments': {'expression': '3*24'}}],
  'answer': 'Order ORD-1047 should arrive in 72 hours.',
  'keys': ['72'],
  'kind': 'two_step'},
 {'user': 'What is the chemical formula for water?',
  'calls': [],
  'answer': 'The chemical formula for water is H2O.',
  'keys': ['h2o'],
  'kind': 'no_tool'}]
```

Training format

`to_chat` turns an example into a full conversation: system, user, assistant tool call, tool result, and so on, ending with the final answer. This is the same message format the agent loop produces at inference time.

```
def to_chat(ex):
    msgs = [{"role": "system", "content": SYSTEM}, {"role": "user", "content": ex["user"]}]
    for c in ex["calls"]:
        msgs.append({"role": "assistant", "content": "", "tool_calls": [{"type": "function", "function": {"name":
c["name"], "arguments": c["arguments"]}}]})
        msgs.append({"role": "tool", "name": c["name"], "content": execute(c)})
    msgs.append({"role": "assistant", "content": ex["answer"]})
    return msgs

print(tok.apply_chat_template(to_chat(train_set[0]), tools=TOOLS, tokenize=False))
```

```
<|im_start|>system
You are a helpful assistant. Use a tool when it is needed; otherwise answer directly.

# Tools

You may call one or more functions to assist with the user query.

You are provided with function signatures within <tools></tools> XML tags:
<tools>
{"type": "function", "function": {"name": "calculator", "description": "Evaluate an arithmetic expression.",
"parameters": {"type": "object", "properties": {"expression": {"type": "string", "description": "e.g. (3+4)*2"}},
"required": ["expression"]}}}
{"type": "function", "function": {"name": "unit_convert", "description": "Convert a value between units (m, km, cm,
mile, ft, inch, kg, g, lb, oz, c, f).", "parameters": {"type": "object", "properties": {"value": {"type":
"number"}, "from_unit": {"type": "string"}, "to_unit": {"type": "string"}}, "required": ["value", "from_unit",
"to_unit"]}}}
{"type": "function", "function": {"name": "get_weather", "description": "Get current weather for a city.",
"parameters": {"type": "object", "properties": {"city": {"type": "string"}}, "required": ["city"]}}}
{"type": "function", "function": {"name": "lookup_order", "description": "Look up an order by its ID (format ORD-
XXXX).", "parameters": {"type": "object", "properties": {"order_id": {"type": "string"}}, "required":
["order_id"]}}}
</tools>

For each function call, return a json object with function name and arguments within <tool_call></tool_call> XML
tags:
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call><|im_end|>
<|im_start|>user
Is it raining in Jaffna?<|im_end|>
<|im_start|>assistant
<tool_call>
{"name": "get_weather", "arguments": {"city": "Jaffna"}}
</tool_call><|im_end|>
<|im_start|>user
<tool_response>
{"temp_c": 33, "condition": "humid"}
</tool_response><|im_end|>
<|im_start|>assistant
It's 33°C and humid in Jaffna.<|im_end|>
```

4. Evaluation

Each test request is run through the full agent loop and scored on the four required metrics. A metric that doesn't apply is stored as `None` and left out of that metric's average.

Metric	Applies to	Counted as correct when
Call validity	tool requests (73)	the model made at least one call, and every call parsed
Call accuracy	tool requests (73)	the right tool(s) with the right arguments. Case, whitespace and number types are normalised, and equivalent calculator expressions are accepted
Abstention	no-tool requests (27)	no tool was called
Task success	all requests (100)	the final answer contains the gold facts, with numbers compared to a 0.01 tolerance

```
NUM_RE = re.compile(r"-?\d[\d,]*\.?\d*")

def _numbers(text):
    out = []
    for m in NUM_RE.findall(text):
        try: out.append(float(m.replace(",", "").rstrip(".")))
        except ValueError: pass
    return out

def _key_ok(key, text):
    t = text.lower()
    for alt in key.lower().split("|"):
        try:
            g = float(alt)
            if any(abs(g - n) <= 0.01 for n in _numbers(t)): return True
        except ValueError:
            if alt.strip() in t: return True
    return False

def _norm_val(v):
    if isinstance(v, bool): return v
    if isinstance(v, (int, float)): return round(float(v), 4)
    if isinstance(v, str):
        s = " ".join(v.strip().lower().split())
        try: return round(float(s), 4)
        except ValueError: return s
    return v

def _call_match(p, g):
    if p["name"] != g["name"] or set(p["arguments"]) != set(g["arguments"]): return False
    if g["name"] == "calculator":
        pe, ge = str(p["arguments"]["expression"]).replace(" ", ""), g["arguments"]["expression"].replace(" ", "")
        return pe == ge or calculator(pe) == calculator(ge)
    return all(_norm_val(p["arguments"][k]) == _norm_val(g["arguments"][k]) for k in g["arguments"])

def score(ex, trace):
    gold, preds = ex["calls"], trace["calls"]
    needs_tool, made_call = len(gold) > 0, len(preds) > 0
    final = trace["final"] or ""
    out = dict(call_valid=None, call_accuracy=None, abstained=None, task_success=None)
    if needs_tool:
        out["call_valid"] = bool(made_call and trace["valid_format"])
        out["call_accuracy"] = bool(out["call_valid"] and len(preds) == len(gold) and all(_call_match(p, g) for p,
g in zip(preds, gold)))
    else:
        out["abstained"] = not made_call
    out["task_success"] = bool(final.strip()) and all(_key_ok(k, final) for k in ex["keys"])
    return out

def evaluate(model, data, name):
    rows = []
    for ex in tqdm(data, desc=name):
        tr = run_agent(model, ex["user"])
        rows.append({**ex, **score(ex, tr), "trace": tr})
    df = pd.DataFrame(rows)
    summary = {m: df[m].dropna().astype(float).mean() * 100 for m in ["call_valid", "call_accuracy", "abstained",
"task_success"]}
    print(name, {k: round(v, 1) for k, v in summary.items()})
    return summary, df
```

Baseline: the untouched model

```
base_summary, base_df = evaluate(model, test_set, "baseline")
base_df.to_pickle(f"{OUT_DIR}/baseline_eval.pkl")
```

```
baseline {'call_valid': np.float64(83.6), 'call_accuracy': np.float64(68.5), 'abstained': np.float64(59.3),
'task_success': np.float64(55.0)}
```

5. LoRA fine-tuning

Only the small LoRA matrices are trained, 8.8 M parameters or 1.75 % of the model. The base weights stay frozen.

The loss is computed on **assistant turns only**. `tokenize_with_assistant_mask` sets the labels of system, user and tool-result tokens to −100, so the model learns to produce tool calls and final answers rather than to predict the prompt.

Training settings: 2 epochs, learning rate 2e-4 with cosine decay, and fp16 mixed precision. The checkpoints and the final adapter are saved to Drive.

```
def tokenize_with_assistant_mask(ex):
    text = tok.apply_chat_template(to_chat(ex), tools=TOOLS, tokenize=False)
    enc = tok(text, truncation=True, max_length=CFG["max_len"], return_offsets_mapping=True)
    labels = [-100] * len(enc["input_ids"])
    for m in re.finditer(r"<\|im_start\|>assistant\n(.*?<\|im_end\|>)", text, re.S):
        s, e = m.start(1), m.end(1)
        for i, (ts, te) in enumerate(enc["offset_mapping"]):
            if te > ts and ts >= s and te <= e:
                labels[i] = enc["input_ids"][i]
    return {"input_ids": enc["input_ids"], "attention_mask": enc["attention_mask"], "labels": labels}

train_tok = [tokenize_with_assistant_mask(ex) for ex in train_set]
```

```
def collate(batch):
    L = max(len(b["input_ids"]) for b in batch)
    pad = lambda x, v: x + [v] * (L - len(x))
    return {
        "input_ids":      torch.tensor([pad(b["input_ids"], tok.pad_token_id) for b in batch]),
        "attention_mask": torch.tensor([pad(b["attention_mask"], 0) for b in batch]),
        "labels":         torch.tensor([pad(b["labels"], -100) for b in batch]),
    }

import gc
for _n in ("trainer", "lora_model", "args"):
    globals().pop(_n, None)
gc.collect(); torch.cuda.empty_cache()

set_seed()
lora_model = get_peft_model(load_model(), LoraConfig(
    r=CFG["r"], lora_alpha=CFG["alpha"], lora_dropout=CFG["dropout"],
    target_modules=CFG["target_modules"], task_type="CAUSAL_LM"))
lora_model.print_trainable_parameters()

args = TrainingArguments(
    output_dir=f"{OUT_DIR}/lora", num_train_epochs=CFG["epochs"],
    per_device_train_batch_size=CFG["batch_size"], gradient_accumulation_steps=CFG["grad_accum"],
    learning_rate=CFG["lr"], lr_scheduler_type="cosine", warmup_steps=5,
    fp16=True, logging_steps=10, save_strategy="epoch", save_total_limit=1,
    report_to="none", seed=SEED,
)
trainer = Trainer(model=lora_model, args=args, train_dataset=train_tok, data_collator=collate)
trainer.train()
lora_model.save_pretrained(f"{OUT_DIR}/lora/adapter")
```

```
trainable params: 8,798,208 || all params: 502,830,976 || trainable%: 1.7497
```

[150/150 06:42, Epoch 2/2]

Step	Training Loss
10	0.340401
20	0.027839
30	0.001664
40	0.002508
50	0.002777
60	0.000812
70	0.000448
80	0.000636
90	0.000303
100	0.000221
110	0.000236
120	0.000207
130	0.000194
140	0.000194
150	0.000253

LoRA model on the same test set

`lora_model.eval()` turns LoRA dropout off. The same 100 requests are then scored exactly as for the baseline.

```
lora_model.eval()
lora_summary, lora_df = evaluate(lora_model, test_set, "lora")
lora_df.to_pickle(f"{OUT_DIR}/lora_eval.pkl")

results = pd.DataFrame({"Baseline": base_summary, "LoRA": lora_summary}).round(1)
results.to_csv(f"{OUT_DIR}/results.csv"); results
```

```
lora {'call_valid': np.float64(100.0), 'call_accuracy': np.float64(90.4), 'abstained': np.float64(96.3),
'task_success': np.float64(92.0)}
```

	Baseline	LoRA
call_valid	83.6	100.0
call_accuracy	68.5	90.4
abstained	59.3	96.3
task_success	55.0	92.0

6. Failure analysis

Each test request gets exactly one category, based on its gold calls and the agent's trace. The checks run in this order:

1. `wrong_format` : a tool call didn't parse.
2. For no-tool requests: `unnecessary_call` (a tool was used) or `wrong_answer` (the direct answer was wrong).
3. `missed_call` : no call was made, or a chain stopped early.
4. `wrong_tool` , then `wrong_args` .
5. `ignored_result` : the calls were correct but the final answer was wrong.
6. Otherwise `ok` .

```
CATEGORIES = ["wrong_format", "wrong_tool", "wrong_args", "unnecessary_call", "missed_call", "ignored_result",
"wrong_answer", "ok"]

def _truthy(x):
    return x is not None and x == x and bool(x)

def categorize(row):
    gold, tr = row["calls"], row["trace"]
    pred_names, gold_names = [c["name"] for c in tr["calls"]], [c["name"] for c in gold]
    if not tr["valid_format"]:
        return "wrong_format"
    if not gold:
        if pred_names: return "unnecessary_call"
        return "ok" if _truthy(row["task_success"]) else "wrong_answer"
    if not pred_names:
        return "missed_call"
    if pred_names != gold_names:
        if gold_names[:len(pred_names)] == pred_names: return "missed_call"
        if pred_names[:len(gold_names)] == gold_names: return "unnecessary_call"
        return "wrong_tool"
    if not _truthy(row["call_accuracy"]):
        return "wrong_args"
    return "ok" if _truthy(row["task_success"]) else "ignored_result"

for df in (base_df, lora_df):
    df["category"] = df.apply(categorize, axis=1)

cmp = pd.DataFrame({"Baseline": base_df["category"].value_counts(), "LoRA":
lora_df["category"].value_counts()}).fillna(0).astype(int)
cmp
```

category	Baseline	LoRA
ignored_result	15	0
missed_call	14	0
ok	44	92
unnecessary_call	11	1
wrong_answer	7	0
wrong_args	3	3
wrong_tool	6	4

What LoRA fixed, left unfixed and introduced

This table pairs each test request's baseline category (rows) with its LoRA category (columns).

```
trans = pd.crosstab(base_df["category"], lora_df["category"], rownames=["baseline"], colnames=["lora"])
fixed      = ((base_df["category"] != "ok") & (lora_df["category"] == "ok")).sum()
not_fixed  = ((base_df["category"] != "ok") & (lora_df["category"] != "ok")).sum()
introduced = ((base_df["category"] == "ok") & (lora_df["category"] != "ok")).sum()
print(f"fixed={fixed}  not fixed={not_fixed}  introduced={introduced}")
trans
```

```
fixed=49  not fixed=7  introduced=1
```

baseline \ lora	ok	unnecessary_call	wrong_args	wrong_tool
ignored_result	15	0	0	0
missed_call	10	0	0	4
ok	43	0	1	0
unnecessary_call	10	1	0	0
wrong_answer	7	0	0	0
wrong_args	1	0	2	0
wrong_tool	6	0	0	0

Failure examples

LoRA produced only 8 failures in total, so all of them are shown. Ten baseline failures are shown for comparison.

```
def show_failures(df, name, n=10):
    print(f"===== {name} failures =====")
    for _, r in df[df["category"] != "ok"].head(n).iterrows():
        print(f"[{r['category']}] {r['user']}\n  gold: {r['calls']} -> {r['answer']}\n  model: {r['trace']
['raw']}\n")

show_failures(base_df, "baseline"); show_failures(lora_df, "lora")
```

```
===== baseline failures =====
[unnecessary_call] What does RAM stand for in computing?
  gold: [] -> RAM stands for Random Access Memory.
  model: ['<tool_call>\n{"name": "lookup_order", "arguments": {"order_id": "RAM"}}\n</tool_call>', 'The term "RAM"
stands for Random Access Memory. It\'s a type of computer memory used to store data temporarily while a program is
running.']

[unnecessary_call] What is the boiling point of water at sea level in Celsius?
  gold: [] -> Water boils at 100 °C at sea level.
  model: ['<tool_call>\n{"name": "unit_convert", "arguments": {"value": 100, "from_unit": "km", "to_unit":
"C"}}\n</tool_call>', 'The boiling point of water at sea level is 100 degrees Celsius.']

[wrong_answer] Who wrote Romeo and Juliet?
  gold: [] -> Romeo and Juliet was written by William Shakespeare.
  model: ["I don't have information about who wrote Romeo and Juliet. I can only provide details about mathematical
operations and weather forecasts. If you need help with writing a story or understanding Romeo and Juliet's plot,
that would be more appropriate for literature experts."]

[wrong_answer] Who wrote Romeo and Juliet?
  gold: [] -> Romeo and Juliet was written by William Shakespeare.
  model: ["I don't have information about who wrote Romeo and Juliet. I can only provide details about mathematical
operations and weather forecasts. If you need help with writing a story or understanding Romeo and Juliet's plot,
that would be more appropriate for literature experts."]

[ignored_result] I placed ORD-1054 last week; what's going on with it?
  gold: [{'name': 'lookup_order', 'arguments': {'order_id': 'ORD-1054'}}] -> Order ORD-1054 (backpack) is
processing; estimated delivery in 6 days.
  model: ['<tool_call>\n{"name": "lookup_order", "arguments": {"order_id": "ORD-1054"}}\n</tool_call>', "The item
'BACKPACK' is expected to be back in stock in 6 days. Please check if you have it before then."]

[ignored_result] I placed ORD-1043 last week; what's going on with it?
  gold: [{'name': 'lookup_order', 'arguments': {'order_id': 'ORD-1043'}}] -> Order ORD-1043 (desk lamp) is
processing; estimated delivery in 5 days.
  model: ['<tool_call>\n{"name": "lookup_order", "arguments": {"order_id": "ORD-1043"}}\n</tool_call>', 'The desk
lamp that was ordered on ORD-1043 will be delivered in approximately 5 days.']

[missed_call] I'm travelling to Ampara - should I pack a coat?
  gold: [{'name': 'get_weather', 'arguments': {'city': 'Ampara'}}] -> It's 4°C and snow in Ampara.
  model: ['To determine if you should pack a coat for your trip to Ampara, we need to know the temperature there.
Could you please provide me with the current temperature in Ampara?']

[ignored_result] Track ORD-1036 for me.
  gold: [{'name': 'lookup_order', 'arguments': {'order_id': 'ORD-1036'}}] -> Order ORD-1036 (water bottle) is
cancelled; estimated delivery in 4 days.
  model: ['<tool_call>\n{"name": "lookup_order", "arguments": {"order_id": "ORD-1036"}}\n</tool_call>', 'The water
bottle at ORD-1036 will be available on day 4 from now until then.']

[wrong_answer] Name a primary colour.
  gold: [] -> Red is a primary colour.
  model: ["I don't have the capability to name colors as they are subjective and not universally recognized.
However, I can help you find information about different primary colours if that would be helpful!"]

[missed_call] Any idea what conditions are like over in Vavuniya today?
  gold: [{'name': 'get_weather', 'arguments': {'city': 'Vavuniya'}}] -> It's 29°C and sunny in Vavuniya.
  model: ["I don't have direct access to real-time weather data. However, I can help you find out if there's any
information about Vavuniya's weather conditions that might be relevant to your question. Could you please provide
me with the date or time of the condition you're interested in?"]

===== lora failures =====
[unnecessary_call] What is the boiling point of water at sea level in Celsius?
  gold: [] -> Water boils at 100 °C at sea level.
  model: ['<tool_call>\n{"name": "calculator", "arguments": {"expression": "0*100"}}\n</tool_call>', 'The boiling
point of water at sea level is 0°C.']

[wrong_args] Take 7 away from 63 - what's left?
  gold: [{'name': 'calculator', 'arguments': {'expression': '63-7'}}] -> The result is 56.
  model: ['<tool_call>\n{"name": "calculator", "arguments": {"expression": "7-63"}}\n</tool_call>', 'The result is
-56.']

[wrong_tool] I'm travelling to Ampara - should I pack a coat?
  gold: [{'name': 'get_weather', 'arguments': {'city': 'Ampara'}}] -> It's 4°C and snow in Ampara.
  model: ['<tool_call>\n{"name": "calculator", "arguments": {"expression": "1*24"}}\n</tool_call>', "It's 24 hours
```

```
before you arrive in Ampara."]

[wrong_tool] I'm travelling to Jaffna - should I pack a coat?
  gold: [{'name': 'get_weather', 'arguments': {'city': 'Jaffna'}}] -> It's 33°C and humid in Jaffna.
  model: ['<tool_call>\n{"name": "calculator", "arguments": {"expression": "1*24"}}\n</tool_call>', "It's 24 hours
until you arrive in Jaffna."]

[wrong_tool] I'm travelling to Jaffna - should I pack a coat?
  gold: [{'name': 'get_weather', 'arguments': {'city': 'Jaffna'}}] -> It's 33°C and humid in Jaffna.
  model: ['<tool_call>\n{"name": "calculator", "arguments": {"expression": "1*24"}}\n</tool_call>', "It's 24 hours
until you arrive in Jaffna."]

[wrong_tool] I'm travelling to Ampara - should I pack a coat?
  gold: [{'name': 'get_weather', 'arguments': {'city': 'Ampara'}}] -> It's 4°C and snow in Ampara.
  model: ['<tool_call>\n{"name": "calculator", "arguments": {"expression": "1*24"}}\n</tool_call>', "It's 24 hours
before you arrive in Ampara."]

[wrong_args] Vavuniya weather report, please.
  gold: [{'name': 'get_weather', 'arguments': {'city': 'Vavuniya'}}] -> It's 29°C and sunny in Vavuniya.
  model: ['<tool_call>\n{"name": "get_weather", "arguments": {"city": "Lisbon"}}\n</tool_call>', "There's no city
named Lisbon in our database."]

[wrong_args] Take 54 away from 76 - what's left?
  gold: [{'name': 'calculator', 'arguments': {'expression': '76-54'}}] -> The result is 22.
  model: ['<tool_call>\n{"name": "calculator", "arguments": {"expression": "54-76"}}\n</tool_call>', 'The result is
-22.']
```

7. Summary

Metric	Baseline	LoRA
Call validity	83.6 %	**100.0 %**
Call accuracy	68.5 %	**90.4 %**
Abstention	59.3 %	**96.3 %**
Task success	55.0 %	**92.0 %**

- **Baseline errors were decisions, not format.** The baseline never produced malformed JSON. It skipped calls it needed (14), called tools for general-knowledge questions (11), refused simple questions (7) and misreported correct tool results (15).
- **LoRA fixed 49 of the 56 baseline failures**, left 7 unfixed and introduced 1. This took 150 training steps, about 7 minutes on a T4.
- **LoRA's remaining errors come from over-fitting to the training templates.** "Should I pack a coat?" is read as the "hours until … arrives" chain, "take *b* away from *a*" is computed as $b - a$, and one weather request used a hallucinated city.